JS- Class-13

28/02/2026

→ We studyin' promises today--

JS → is single threaded means

- Only one thing happens at a time.
- One call stack → one execution at a time
- Like a single waiter taking one order at a time.

Problem

↓

If a task takes too long -- everything blocks!

Solution

Outsourcing

```
→ console.log("start")
→ for (let i=0 ; i<100000000 ; i++) {}
→ console.log("ends")      //waits for loop to end..
```

→ Javascript doesn't handle everything itself

It "OUTSOURCES" tasks to web APIs (browser) or C++ APIs (Node.js)

Who handles what :

Call stack ← JS engine handles this

web APIs ← browser handles (setTimeout, DOM, AJAX)

Callback Queue ← Waiting area for completed tasks

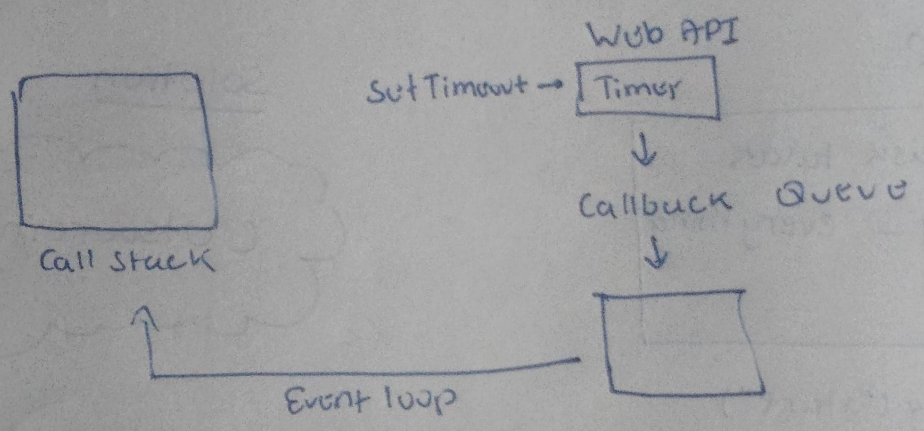
Event Loop ← Traffic controller

Event loop

What goes inside call stack → Regular JS code exec

Event loop's JOB:

- Check if call stack is empty
- If empty, takes first task from callback Queue.
- Pushes it to Call stack
- REPEAT forever



Promises : An object representing:

"The eventual completion (or failure) of an asynchronous operation"

e.g restaurant---

you get the recipt first

but your food comes later

can be Pending / Resolved / REJECT

(by default)

```
const promise = new Promise (Creolve, reject) => {
        //do something asynchronous here

        if (/* everthing succesful */) {
            resolve ("Success value");
        }
        else {
            reject ("Error mussage");
        }
    });
```

// 'resolve' and 'reject' are functions given to you by javascript

// You don't define them.. you jus CALL them!

1'st parameter → resolve

2nd parameter → reject

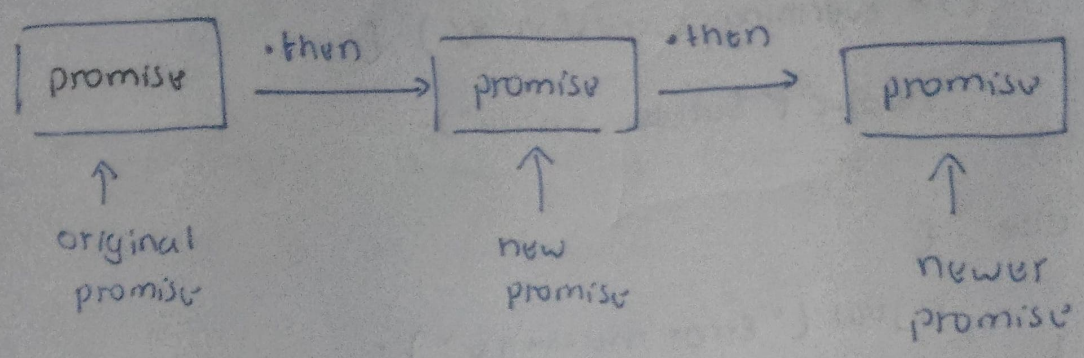
```
promise.then (
    (value) => console.log (data),
    (error) => console.log (error)..        // ho gaya error handle..  2nd parm
);
```

```
promise.then (
    (value) => console.log ( data)
). catch ((error) => console-log (error));
```

.then → values ko aagey pas kar sakta hai...

Every .then() and .catch() returns a new Promise

This is what makes CHANGING possible.



VALUE FLOW IN CHAINS

value gets passed down the chain --

```
Promise.resolve(10)
    .then(value => {
        console.log(value);        // 10
        return value * 2;          // pass 20 to next
    })
    .then(value => {
        console.log(value);        // 20
        return value + 5;          // pass 25 to next
    })
    .then(value => {
        console.log(value);        // 25
    });
```

CATCH ISN'T ALWAYS NEEDED AT END

Sabse zyada istemaal..

↓

fetch

returns a ref (promise)

object = {
state = pending | res | rej
thenArray [fn, fn2 ...]
catchArray [fn ...]
}

Jo bhi data aayega ...
then and catch Array me bhej do

JS Ext tool
- fetch
- Timers
- console
- DOM

OS → network Call

Instant resolve..

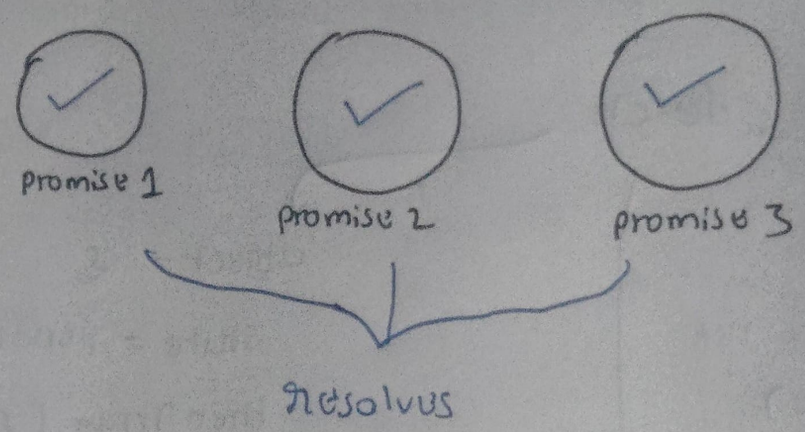
```
const inst = Promise.resolve("Instantly resolve");
console.log(inst); // Promise {"Turant"}
```

Combinators

Static methods that work with multiple promises simultaneously

Promise.all	ALL promise resolve	ANY promise rej
Promise.allsettled	All promise settle (resolve/rej)	Never reject
Promise.race	first promise settle	first promise rej
Promise.any	first promise resolves	ALL promises rej

1. promise.all ( ) - wait for All



2. promise.allSettled ( ) - wait for all -- don't care abt failures

```
Const promises = [
        fetch ('/critical-data'),
        fetch ('/optional-data'),
        fetch ('/analytics')
];

Promise.allSettled (promises)
    .then ( results => {
        results.forEach ((result, idx) => {
            if (result.status === 'fulfilled') {
                console.log (`Promise ${index} suceed`)
            } else {
                console.log (`Promise ${index} failed`)
            }
        });
    });
```

// No .catch() → It never rejects!

3. promise.race() - "first one wins (settlement)"

whatever happens first--

resolve, reject-- → it happens

4. Promise.any() - "first success wins"

if any resolves → resolves..

if all fail → fail